

Guía de aprendizaje — Backend Fullstack con Flask + JWT

Objetivo del proyecto

El objetivo de este proyecto es aprender cómo funciona una aplicación web real desde cero:

- Backend con Flask
- Base de datos
- CRUD de usuarios
- Autenticación con JWT
- Frontend conectado al backend
- Manejo de perfiles
- Seguridad básica
- Buenas prácticas

Este proyecto sirve como:

- práctica personal
- portfolio junior
- base para proyectos más grandes
- preparación para entrevistas

FASE 1 — Entender la arquitectura

¿Qué es frontend?

El frontend es la parte visual:

- HTML
- CSS
- Tailwind
- JavaScript

Se encarga de:

- mostrar información
 - enviar formularios
 - consumir APIs
 - mostrar mensajes
-

¿Qué es backend?

El backend es el servidor.

Se encarga de:

- lógica
- autenticación
- base de datos
- seguridad
- respuestas JSON

En este proyecto se utilizó:

- Flask
 - SQLAlchemy
 - JWT
-

¿Qué es una API?

Una API permite comunicación entre frontend y backend.

Ejemplo:

Frontend:

```
fetch('/users')
```

Backend:

```
@main.route('/users')
def get_users():
```

FASE 2 — Configuración del backend

Crear entorno virtual

¿Por qué?

Para aislar dependencias del proyecto.

Comando:

```
python -m venv venv
```

Activar:

Windows:

```
venv\Scripts\activate
```

Instalar Flask

```
pip install flask
```

Instalar Flask SQLAlchemy

```
pip install flask_sqlalchemy
```

¿Por qué?

Permite trabajar con base de datos usando Python.

Instalar JWT

```
pip install flask-jwt-extended
```

¿Por qué?

Para autenticación moderna usando tokens.

FASE 3 — Arquitectura del proyecto

Estructura utilizada

```
api_usuarios/  
├── app/  
│   ├── __init__.py  
│   ├── models.py  
│   └── routes.py  
├── venv/  
├── config.py  
├── run.py  
└── requirements.txt
```

¿Por qué separar archivos?

Porque en proyectos reales:

- modelos
- rutas
- configuración
- lógica

no deben estar mezclados.

Esto mejora:

- mantenimiento
- escalabilidad
- legibilidad

FASE 4 — Crear la aplicación Flask

create_app()

```
from flask import Flask
```

```
def create_app():  
    app = Flask(__name__)  
  
    return app
```

¿Por qué usar create_app?

Porque es una arquitectura profesional.

Permite:

- modularidad
- testing
- escalabilidad
- múltiples configuraciones

FASE 5 — Modelos y base de datos

Modelo User

```
class User(db.Model):
```

Representa una tabla de usuarios.

Campos básicos

```
id = db.Column(db.Integer, primary_key=True)  
name = db.Column(db.String(100))  
password = db.Column(db.String(255))
```

¿Por qué usamos modelos?

Porque SQLAlchemy convierte objetos Python en tablas SQL.

Ejemplo:

```
user.name
```

se convierte en:

```
SELECT name FROM users
```

FASE 6 — CRUD

CRUD significa:

- Create
- Read
- Update
- Delete

CREATE

Endpoint POST

```
@main.route('/users', methods=['POST'])
```

Permite crear usuarios.

READ

Obtener usuarios

```
@main.route('/users', methods=['GET'])
```

Obtener usuario por ID

```
@main.route('/users/<int:user_id>', methods=['GET'])
```

UPDATE

Actualizar usuario

```
@main.route('/users/<int:user_id>', methods=['PUT'])
```

DELETE

Eliminar usuario

```
@main.route('/users/<int:user_id>', methods=['DELETE'])
```

FASE 7 — Seguridad de contraseñas

Problema

Nunca se deben guardar contraseñas normales.

✗ Incorrecto:

```
password = '123456'
```

Solución: hashing

```
generate_password_hash()
```

Verificación

```
check_password_hash()
```

¿Por qué?

Porque si la base de datos es comprometida:

- las contraseñas reales no quedan expuestas
-

FASE 8 — Login

Endpoint login

```
@main.route('/login', methods=['POST'])
```

Flujo del login

1. frontend envía username/password
 2. backend busca usuario
 3. backend verifica hash
 4. backend crea JWT
 5. frontend guarda token
-

JWT

```
create_access_token()
```

¿Qué contiene el token?

En este proyecto:

```
identity=str(user.id)
```

FASE 9 — Rutas protegidas

jwt_required

```
@jwt_required()
```

Protege endpoints.

get_jwt_identity

```
get_jwt_identity()
```

Obtiene el ID del usuario autenticado.

¿Por qué es importante?

Permite:

- autenticación
 - autorización
 - seguridad
-

FASE 10 — Cambio de contraseña

Endpoint especializado

```
/change-password
```

¿Por qué separado?

Porque cambiar contraseña es una acción sensible.

Buenas prácticas:

- endpoint separado

- validar contraseña actual
 - usar hashing
-

Flujo

1. usuario escribe current password
 2. escribe nueva password
 3. backend valida contraseña actual
 4. backend guarda nueva contraseña
-

FASE 11 — Frontend

Objetivo

Consumir la API.

Tecnologías

- HTML
 - TailwindCSS
 - JavaScript
-

fetch

```
fetch('http://127.0.0.1:5000/users')
```

Authorization header

```
Authorization: 'Bearer ' + token
```

¿Por qué guardar token?

Para mantener sesión iniciada.

FASE 12 — Organización frontend

Estructura

```
frontend/  
|  
├─ pages/  
├─ services/  
├─ utils/  
└─ components/
```

pages

Contiene lógica de páginas.

Ejemplo:

```
profile.js  
login.js
```

services

Contiene llamadas fetch.

Ejemplo:

```
getUsuarioPorId()
```

utils

Funciones reutilizables.

Ejemplo:

```
renderUsuario()
```

FASE 13 — Profile page

Objetivo

Crear una página real de perfil.

Funciones:

- editar username
 - cambiar password
 - avatar dinámico
 - confirm password
 - current password
 - toast notifications
-

Avatar dinámico

Se generan iniciales automáticamente.

Ejemplo:

```
John Doe → JD
```

toggleEdit()

Permite:

- activar edición
 - bloquear inputs
 - mostrar/ocultar campos
-

saveChanges()

Se encarga de:

- detectar cambios
 - llamar endpoints correctos
 - enviar fetch
 - mostrar toast
-

Toast

Mensaje temporal visual.

Ejemplo:

✓ Cambios guardados correctamente

FASE 14 — Seguridad

Problemas detectados

Inicialmente:

- endpoints sin JWT
 - cualquiera podía editar usuarios
-

Solución

Agregar:

@jwt_required()

Authorization

Verificar:

```
if str(user_id) != current_user_id:
```

Diferencia importante

Authentication

¿Quién eres?

JWT.

Authorization

¿Qué puedes hacer?

Validar permisos.

FASE 15 — Lo aprendido

Backend

- Flask
 - SQLAlchemy
 - JWT
 - hashing
 - CRUD
 - APIs REST
 - rutas protegidas
 - validaciones
-

Frontend

- fetch
 - consumo APIs
 - manejo DOM
 - módulos JS
 - render dinámico
 - UX básica
-

Arquitectura

- separación de responsabilidades
 - modularidad
 - estructura profesional
-

FASE 16 — Próximos pasos

Prioridad inmediata

Terminar profile page

- guardar username
 - guardar password
 - mensajes de error
 - limpiar inputs
-

Siguiente gran paso

Roles

Agregar:

`admin`
`user`

Permisos

Admin:

- ver usuarios
- eliminar usuarios

User:

- editar perfil
 - borrar cuenta propia
-

Dashboard

Crear:

- tabla usuarios
 - logout
 - protección de rutas
 - redirecciones
-

Deploy

Aprender:

- Render
 - Railway
 - Vercel
-

Meta final

Construir una aplicación completa con:

- login
 - JWT
 - roles
 - CRUD
 - dashboard
 - frontend moderno
 - backend seguro
-

Conclusión

Este proyecto ya incluye muchos conceptos reales utilizados en aplicaciones modernas.

Lo más importante aprendido hasta ahora:

- cómo frontend y backend se comunican
- cómo funciona autenticación
- cómo proteger rutas
- cómo organizar proyectos
- cómo manejar usuarios reales

- cómo construir APIs

Este proyecto puede evolucionar a:

- sistema administrativo
- SaaS
- panel empresarial
- portfolio profesional
- base para entrevistas junior